

Von Hand zu digital - und jetzt auf der *anderen Seite* des Tisches.

Entwicklung einer Software zur Lagerverwaltung von medizinischem Verbrauchsmaterial im Krankenhaus



Sascha Schulz

Wirtschaftsinformatiker • Betreuerin: Frau Cramer • Java + JavaFX + MySQL

Wer hier spricht

Drei Jahre Teamleiter im Zentrallager – belegt im Arbeitszeugnis

Bevor ich Software entwickelt habe, habe ich Lager **geführt**. Zuletzt als Teamleiter im Zentrallager der Diakonissen Speyer (02/2020–03/2023) – genau das Umfeld, das dieses Projekt im Kleinen nachbildet.



Diakonissen Speyer

1.200

lagergeführte Artikel

24.000+

Artikel über den Einkauf

15.000

Lieferscheine / Jahr

12

Mitarbeiter geführt

Zahlen aus der beruflichen Tätigkeit; Warenwirtschaft im Unternehmen: mySAP ERP / Modul MM (laut Arbeitszeugnis Diakonissen Speyer mbH).



Die Ausgangslage - damals

Nur das Zentrallager hatte ein System. Der Rest lief auf Papier.



✗ Die Schwachstellen

Medienbruch Papier→System • Tippfehler & Doppelerfassung • keine Echtzeit-Bestände • jede Bestellung einzeln • Stationen ohne Systemzugang



DAMALS 13 Außenhäuser + Stationen bestellen per Fax



HEUTE jeder Standort bestellt direkt im System

Die Wende: Pilotprojekt Amondis

Federführend bei Anpassung, Zuschnitt und Einführung der Software

VORHER

Insel-Lösung

Nur das Zentrallager arbeitete im SAP. Stationen & Außenhäuser schickten Zettel – zentral abgetippt.

- ✗ Manuelle Erfassung im Zentrallager
- ✗ Bestellungen einzeln & unverbunden
- ✗ kein direkter Standort-Zugriff



NACHHER

Voll digitaler Ablauf

Zentrallager, alle Stationen und jedes Außenhaus erhielten Amondis – passend zugeschnitten, bestellen direkt.

- ✓ Jeder Standort bestellt selbst digital
- ✓ Mehrfach-Bestellungen zusammengefasst
- ✓ durchgängiger Prozess ohne Papier



Neue Funktion: automatische Trennung – vorher nicht möglich

~1.200 **lagergeführte Artikel** verbleiben im Lager · **24.000+ Artikel** werden automatisch an den Einkauf zur Bestellung weitergeleitet. Das System trennt beide Wege selbstständig – diese Aufteilung war früher nicht möglich.



DAMALS Amondis anpassen & verteilen (Anwenderseite)



HEUTE Lagersoftware selbst entwerfen & bauen

Der Brückenschlag

Warum genau dieses Projekt?



DAMALS

Anwender

Ich habe die Software bedient, angepasst und eingeführt.



HEUTE

Entwickler

Ich baue das System selbst – Datenmodell, Logik und Oberfläche.

„Ich kenne den Schmerz der Zettelwirtschaft aus erster Hand. Jetzt löse ich ihn als Entwickler – das ist meine Motivation für dieses Projekt.“

Das Projekt

Dieselbe Aufgabe – jetzt von der Entwicklerseite gelöst



Auftrag

Lagerverwaltung medizinischen Verbrauchsmaterials für ein (fiktives) Krankenhaus.



Ziel

Manuelle Listen ablösen durch eine strukturierte Desktop-Anwendung mit Datenbank.



Umfang

CRUD über 6 Tabellen, Suche/Filter, Bestands- und Bestellverwaltung, lokal ohne Internet.

Synonym zur Praxis

Krankenhaus mit Lager

Verbrauchsmaterial verwalten

Bestand & Bestellungen

Mindestbestand statt ABC-Liste

Mehrere Lager / Stationen

wie Zentrallager + Außenhäuser



DAMALS reales Krankenhauslager mit SAP/Amondis



HEUTE fiktives Lager – eigene Java-Anwendung

Die Anforderungen

SRH-Pflichtkriterien – jede spiegelt eine reale Lager-Aufgabe



MySQL-Datenbank

mind. 5 Tabellen → umgesetzt mit 6 Tabellen (3NF)



CRUD-Funktionalität

Anlegen, Lesen, Ändern, Löschen aller Stammdaten



MVC-Architektur

saubere Trennung von Daten, Logik und Oberfläche



Such- & Filterfunktion

Material schnell finden – wie die Kommissionierung



DAMALS SAP/Amondis erfüllten diese Aufgaben im Großen



HEUTE selbe Bausteine – im Kleinen selbst gebaut

Das Problem im Modell

Die Ist-Analyse des Krankenhauses spiegelt die Insel-Situation von früher



Manuelle Listen

Bestände in Excel/Papier – fehleranfällig und nie aktuell.



Kein Warnsystem

Niemand merkt rechtzeitig, wenn ein Material zur Neige geht.



Keine Historie

Zu- und Abgänge sind nicht nachvollziehbar dokumentiert.



Soll-Konzept

Eine zentrale Anwendung mit Datenbank: jederzeit aktuelle Bestände, automatische Mindestbestand-Warnung und lückenlose Bewegungs-Historie – genau das, was Amondis im echten Lager geschaffen hat.



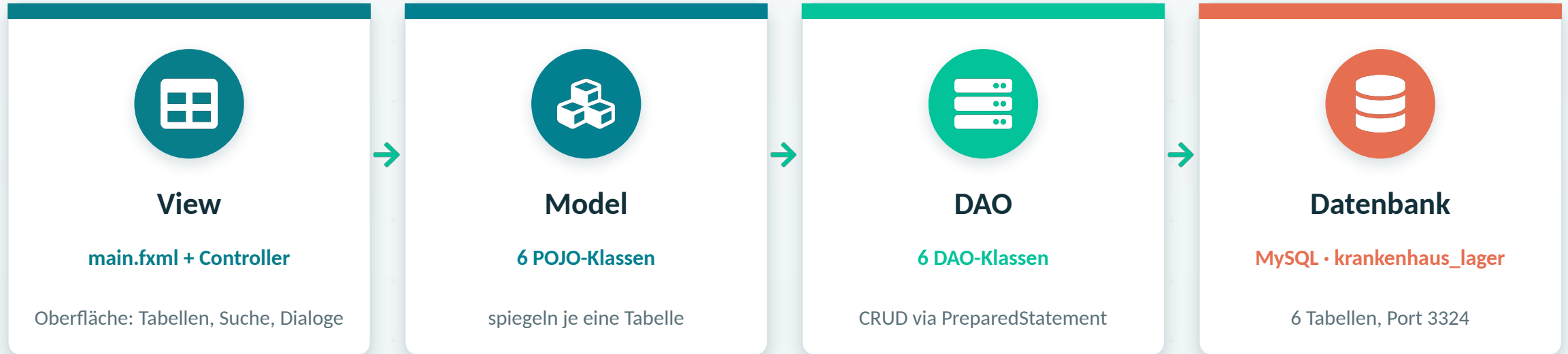
DAMALS Zettel, kein Warnsystem, keine Nachvollziehbarkeit



HEUTE DB-gestützt, Warnungen, volle Historie

Die Lösung: Architektur

MVC + DAO-Muster • Java 17 • JavaFX • MySQL 8 – lokal, ohne Internet

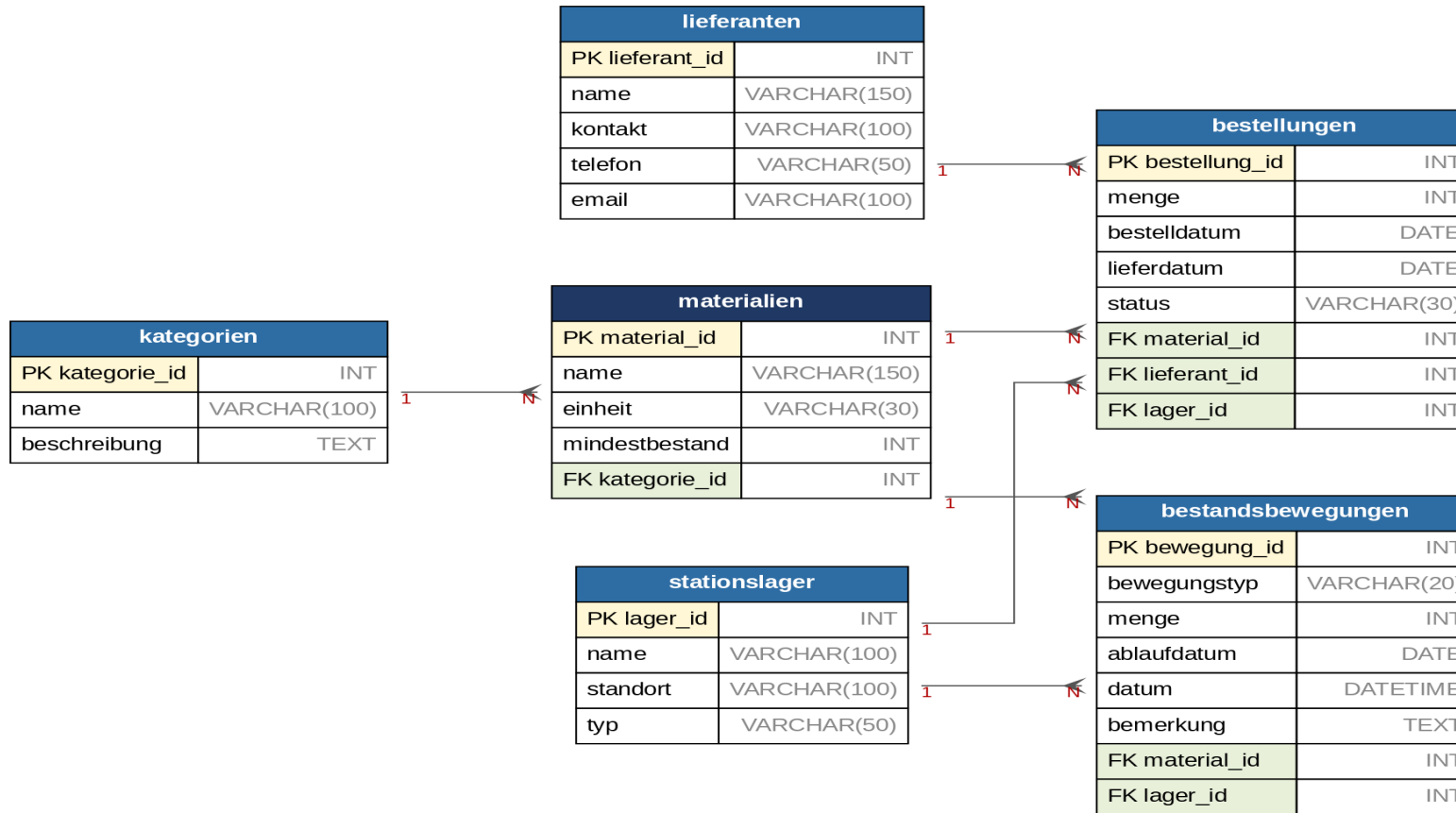


Wie ein Warenwirtschaftssystem im Kleinen – nur dass ich jede Schicht selbst entworfen habe.

Das Datenmodell

6 Tabellen in 3. Normalform – das Herz der Anwendung

ER-Diagramm: Materialverwaltung Krankenhaus (6 Tabellen, 3NF)



Die 6 Tabellen

- **kategorien** Materialgruppen
- **materialien** Artikelstammdaten
- **stationslager** Lagerorte / Stationen
- **lieferanten** Bezugsquellen
- **bestellungen** offene Bestellungen
- **bestandsbewegungen** Ein-/Ausgänge



Die Kernfunktionen

Was die Anwendung kann – jede Funktion mit Praxisbezug



CRUD über alle Daten

Stammdaten anlegen, ändern, löschen



Suche & Filter

Material nach Name oder Station finden



Mindestbestand-Warnung

kritische Bestände rot hervorgehoben



Umlagerung

Material zwischen Lagern umbuchen (Bonus)



Bestellverwaltung

offene Bestellungen mit Status



Bestandsberechnung

Bestand live aus den Bewegungen



Designentscheidung im Detail

Der Bestand wird nicht gespeichert – er wird berechnet.

$$\text{Bestand} = \Sigma \text{EINGANG} - \Sigma \text{AUSGANG}$$

Warum? Jede Buchung ist ein Datensatz in **bestandsbewegungen**. Der aktuelle Bestand entsteht per SQL-Aggregation (SUM / CASE WHEN). Das verhindert widersprüchliche Werte und liefert kostenlos eine lückenlose Historie – dasselbe Prinzip wie die bestandsgeführte Logik in SAP.

→ Eine bewusste Entscheidung, die ich im Fachgespräch begründen kann.

Herausforderungen & Lösungen

Aus dem Projekttagbuch – dieselbe Systematik wie bei der Prozessanpassung



Falscher Spaltenname

PK hieß im Code anders als in der DB → SQLException → **an allen Stellen vereinheitlicht**



DB-Verbindung scheiterte

falscher Datenbankname & Port in der JDBC-URL → **auf krankenhaus_lager / 3324 korrigiert**



JavaFX selbst erarbeitet

TableView, FilteredList, FXML waren nicht im Unterricht → **über Eigenrecherche + Lernhandbuch gelöst**



DAMALS Amondis-Oberflächen im Betrieb anpassen



HEUTE Code analysieren, Fehler isolieren, lösen

Das Ergebnis

Alle Pflichtenorderungen erfüllt – plus eine Zusatzfunktion

6

Tabellen (Soll: ≥5)

~1.700

Zeilen Java-Code

15+

Klassen (MVC/DAO)

+1

Bonus: Umlagerung

- ✓ MySQL-Datenbank mit 6 Tabellen (3NF)
- ✓ MVC-Architektur mit DAO-Schicht
- ✓ Mindestbestand-Warnung in Rot
- ✓ Vollständiges CRUD über alle Entitäten
- ✓ Such- und Filterfunktion
- ✓ Umlagerung als zusätzliches Feature



Vom Fax-Zettel zum eigenen System

Ich habe die Digitalisierung eines Krankenhauslagers als **Anwender** begleitet – und sie hier als **Entwickler** von Grund auf neu gedacht.



Praxis verstanden

Ich kenne das Problem aus dem echten Lageralltag.



Selbst umgesetzt

Datenmodell, Logik und Oberfläche eigenständig gebaut.



Ziel erreicht

Alle SRH-Anforderungen erfüllt, plus Zusatzfeature.

Vielen Dank – ich freue mich auf Ihre Fragen.